SpecSplit: An Exploration of Disaggregated Speculative Decoding

Amit Namburi
UC San Diego
anamburi@ucsd.edu

Andrew Yang
UC San Diego
any013@ucsd.edu

Arul Mathur
UC San Diego
armathur@ucsd.edu

Divyam Sengar
UC San Diego
dsengar@ucsd.edu

Samvrit Srinath
UC San Diego
sasrinath@ucsd.edu

1 Introduction

1.1 Background and Motivation

The trajectory of Large Language Model (LLM) development is governed by a relentless scaling dynamic: capability scales with parameter count [8]. However, this growth has produced a fundamental hardware crisis known as the *Memory Wall*. Autoregressive token generation is strictly memory-bandwidth bound. A model such as Llama-3-70B must transfer its full 140 GB of weights from High Bandwidth Memory (HBM) to on-chip SRAM for every single token it produces [2]. The GPU's compute cores—capable of 312 TFLOPS on an A100—sit largely idle during this transfer, yielding actual compute utilization as low as 10% during the decode phase [15]. The bottleneck is not intelligence; it is memory.

This physical constraint creates a prohibitive economic barrier. In the current monolithic serving paradigm, deploying a 70B model demands enterprise-class GPUs solely to satisfy memory capacity requirements. An A100-80GB runs \$1.10–\$4.10 per hour [17], irrespective of whether its compute cores are busy. The result is a *VRAM Tax*: users pay for intelligence but wait on memory. For students, independent researchers, and practitioners at smaller organizations, the options are grim. Aggressive quantization degrades reasoning fidelity; CPU offloading produces unusable generation rates of 1–2 tokens per second [22]; closed-source API access surrenders control over model behavior, fine-tuning, and data privacy. No viable path currently exists for running uncompressed, full-quality 70B-scale models at interactive speeds without enterprise-level hardware costs.

1.2 The Core Insight: Disaggregated Speculative Decoding

We propose **SpecSplit**, an exploratory study into *Disaggregated Speculative Decoding*. Rather than coupling draft and target models on the same physical machine—the standard assumption of all prior speculative decoding work—SpecSplit decouples the two phases onto physically separate hardware tiers connected via commodity cloud networking. A cheap commodity GPU runs the small draft model continuously, while a premium GPU performs only the brief, parallelizable verification step.

The viability of this decomposition rests on a concrete latency inequality. A 70B parameter target model on a single A100 exhibits a Time-Per-Output-Token of 30–50 ms, imposed by the memory-bandwidth bottleneck [18, 20]. The intra-region Round Trip Time between instances in the same cloud availability zone is consistently below 2 ms [7, 9]. This yields a roughly 15–17$\times$ differential:

$$T_{\text{network}}\ (\approx 2\,\text{ms}) \ll T_{\text{verify}}\ (\approx 30\text{–}50\,\text{ms}) \tag{1}$$

Because SpecSplit transmits only integer token IDs and scalar log-probabilities—never hidden states or weight tensors—each round's network payload is under 10 KB regardless of model size. This keeps the transfer cost negligible relative to the verification window, making the disaggregation structurally free in the asymptotic case.

1.3 Scope: A Feasibility Exploration

We deliberately frame this work as an *exploration* rather than a production system. Every component of SpecSplit—the KV cache management, the gRPC transport layer, the inference loop, the verification logic, and the asynchronous pipeline—was built from scratch as part of this project. We did not integrate with existing serving frameworks such as vLLM [10] or TensorRT-LLM. The goal was not to build the fastest possible implementation but to answer a prior question: *is disaggregated speculative decoding even feasible over commodity networking, and what are the practical constraints that govern its behavior?*

Our system is not optimized for throughput; it is instrumented for measurement. We prioritize observability: per-round telemetry, timing breakdowns, hit/miss accounting—over raw performance, because the primary contribution is the feasibility characterization itself. Where production systems would hide latency through kernel fusion and batching, we instead surface it to understand where it comes from.

1.4 Consumer Impact and Model Heterogeneity

SpecSplit has two concrete implications for access-democratization. First, it enables *model heterogeneity by design*. The draft and target models need not belong to the same family, provided they share a tokenizer vocabulary. Verification operates in shared token space: only token

IDs and log-probabilities cross the network—so any pair of vocabulary-compatible models can be composed. This opens a marketplace-style topology: a user runs a cheap commodity GPU for continuous generation throughput while renting a premium GPU for milliseconds of verification intelligence per output token.

Second, SpecSplit enables a *rent-by-the-millisecond* model for the verification resource. Because a single verification pass lasts $\sim$35 ms, the target GPU is active for only a fraction of wall time. A single A100 could in principle serve multiple independent drafting clients with time-multiplexed verification, amortizing its cost across users rather than requiring each user to provision a dedicated enterprise session. These economic properties are analyzed in Section 3.

1.5 Contributions

This paper makes the following contributions as an exploration of the disaggregated speculative decoding paradigm:

- **End-to-End Prototype:** We design and implement SpecSplit from scratch—including KV cache management, gRPC transport, inference loops, and verification logic—as a self-contained prototype targeting commodity cloud hardware. The full system is released as open-source.
- **Feasibility Analysis:** We characterize the practical conditions under which network-disaggregated speculative decoding is viable, grounding the analysis in the Feasibility Inequality (Equation 1) and validating it empirically across two hardware harnesses.
- **Asynchronous Latency Hiding:** We design an overlapped pipeline that concurrently generates the next draft tree while the current tree is being verified, converting the network RTT from a serial cost into a hidden latency.
- **Empirical Characterization:** We evaluate the system across 62 prompts spanning six task categories, identifying acceptance rate, speculation hit rate, and workload entropy as the primary drivers of pipeline efficiency, and providing a benchmark harness for systematic future evaluation.

2 Related Work

2.1 Standard Speculative Decoding

Autoregressive decoding is memory-bandwidth bound, causing severe underutilization of GPU compute. Speculative decoding addresses this by using a small, efficient draft model to generate candidate tokens that a larger target model then verifies in a single parallel forward pass. The seminal formulation, introduced concurrently by Leviathan et al. [11] and Chen et al. [5], guarantees output equivalence to the target model's distribution via rejection sampling, while achieving substantial speedups by converting otherwise idle target compute into useful verification throughput. Both works assume the draft and target reside on the same machine, with no consideration of inter-node communication.

2.2 Tree-Based and Feature-Level Speculation

Extending linear drafting to tree structures improves acceptance rates by exploring multiple candidate continuations simultaneously. SpecInfer [16] organizes draft tokens into a token tree verified in a single batched forward pass, yielding 1.5–2.8$\times$ speedups over standard serving. Medusa [4] takes a different approach, attaching auxiliary prediction heads directly to the target model so that multiple future tokens are predicted in parallel without a separate draft model. Sequoia [6] extends tree speculation further with a dynamic programming algorithm that finds hardware-aware optimal tree configurations for a given GPU platform, demonstrating that tree structure selection is itself a tunable systems parameter.

At the feature level, EAGLE [12] autoregressively drafts at the target's penultimate hidden-state layer rather than in token space, producing candidates substantially more aligned with the target distribution and achieving 2.7–3.5$\times$ speedups. EAGLE-2 [13] adds dynamic, context-dependent draft trees. A critical shared assumption across all of these systems—Medusa, Sequoia, SpecInfer, EAGLE—is *hardware co-location*: the draft and target models share the same machine, communicate through local memory or NVLink, and require no network-aware design. SpecSplit breaks this assumption entirely. In doing so it sacrifices the feature-level drafting advantages of EAGLE (which requires access to the target's hidden states) but gains hardware independence and the ability to compose any vocabulary-compatible model pair.

2.3 Heterogeneous-Hardware Speculation

Several recent systems disaggregate speculation across physically distinct accelerators. Dovetail [22] assigns the draft model to a consumer-grade GPU and the target model to a CPU, demonstrating that complementary throughput characteristics can be exploited even when the target runs on the slower device. Mirror-SD [3] maps draft and target computation across GPU and NPU accelerators on Apple SoC hardware, overlapping their execution to achieve 2.8–5.8$\times$ wall-time speedups. Both explore hardware asymmetry but target *co-located* heterogeneous devices with shared high-bandwidth interconnects—a fundamentally different regime from SpecSplit's geographically separated cloud instances connected only by commodity Ethernet.

The closest architectural parallel is SwiftSpec [23], which redesigns speculative decoding as an asynchronous disaggregated pipeline where draft and target scale independently. SwiftSpec serves Llama-3-70B at 348 tokens/s on 8 NVIDIA Hopper GPUs over NVLink/InfiniBand, removing the draft from the target's critical path in a manner similar to SpecSplit's overlapped pipeline. The key distinction is hardware regime: SwiftSpec targets proprietary enterprise interconnects, while SpecSplit targets standard Ethernet with no specialized networking. Our feasibility analysis (Section 3) is specifically concerned with whether SwiftSpec's logical decomposition remains viable when the interconnect assumption is removed.

2.4 Production LLM Serving Frameworks

A separate but relevant body of work addresses high-throughput LLM serving at scale. vLLM [10] introduced PagedAttention, a KV cache management scheme that virtualizes GPU memory to enable continuous batching and near-zero memory waste. It has become a standard reference serving system against which throughput improvements are benchmarked. TensorRT-LLM provides compiler-level kernel fusion and quantization for NVIDIA hardware, achieving near-peak hardware utilization for co-located serving. Ray Serve provides the distributed orchestration layer that production deployments use to schedule requests across multi-GPU clusters.

SpecSplit does not integrate with any of these frameworks. Our KV cache management, inference loop, and serving logic are custom-built from scratch. This is a deliberate scope decision—building on vLLM or TensorRT-LLM would have obscured the protocol-level questions we are trying to answer—but it also means SpecSplit's raw throughput is not directly comparable to these systems. The relevant comparison is not "SpecSplit vs. vLLM on tokens/s" but rather "does disaggregated speculation over commodity networking introduce prohibitive overhead relative to the verification window?"

2.5 Prefill-Decode Disaggregation

Orthogonal to speculative decoding, DistServe [24] and Splitwise [19] disaggregate the prefill and decode phases of standard autoregressive generation onto separate physical machines. They recognize that prompt processing is compute-bound while token generation is memory-bandwidth bound, and that co-locating them causes harmful interference. SpecSplit applies the same disaggregation intuition but at a different boundary—draft vs. verify rather than prefill vs. decode—and in a fundamentally different infrastructure regime: commodity cloud instances over standard Ethernet rather than enterprise clusters with InfiniBand. CacheGen [14] explores KV cache compression and streaming for distributed serving, a direction directly relevant to reducing the per-session synchronization overhead that SpecSplit's independent draft/target caches currently incur.

2.6 SpecSplit in the Landscape

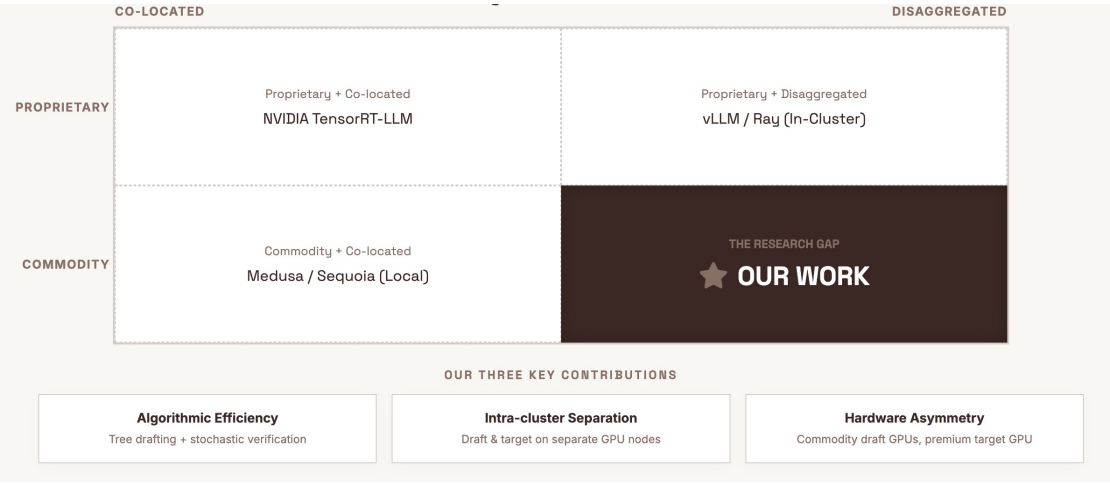


Figure 1: The LLM serving landscape along two axes: hardware tier (commodity vs. proprietary) and deployment topology (co-located vs. disaggregated). SpecSplit occupies the underexplored commodity-disaggregated quadrant, which existing frameworks either do not target or explicitly assume away.

The systems described above can be organized along two axes: *hardware tier* (commodity vs. proprietary/enterprise) and *deployment topology* (co-located vs. disaggregated). This produces four quadrants, illustrated in Figure 1.

Proprietary + Co-located describes the dominant paradigm: TensorRT-LLM, Medusa, EAGLE, and Sequoia all assume high-end hardware with the draft and target on the same machine or rack, communicating through NVLink or shared memory. **Proprietary + Disaggregated** describes DistServe, Splitwise, and SwiftSpec, which use InfiniBand interconnects to disaggregate phases across enterprise clusters. **Commodity + Co-located** includes Sequoia in its local-hardware configuration and Dovetail, which run speculative decoding on consumer-grade hardware without network separation.

Commodity + Disaggregated—standard hardware, standard networking, physically separated workers—is the quadrant SpecSplit occupies. No existing system explicitly targets this combination. The standard systems that come closest (SwiftSpec, DistServe) assume proprietary interconnects; the commodity systems (Sequoia local, Dovetail) assume co-location. SpecSplit's contribution is to characterize whether the commodity-disaggregated quadrant is feasible at all, and to identify the practical constraints—acceptance rate, miss penalty, network variance—that govern its behavior.

3 Motivation and Challenges

3.1 The Access Problem

The practical barrier to running large language models is not algorithmic—it is economic. A student or independent researcher who wishes to run an unquantized 70B model at interactive speeds faces a discrete cliff: they must provision an A100-class GPU (80 GB VRAM)

continuously for the duration of their session, accept significant quality degradation through quantization, or tolerate generation rates of 1–2 tokens per second via CPU offloading [22]. There is no intermediate option.

This is not a consequence of the model being computationally expensive. A 70B model requires moving $\sim$140 GB of weights per token generated, but the floating-point computation per token is modest. The A100's 312 TFLOPS of compute go essentially unused—approximately 10% of peak compute is utilized during decode [15]. The user is paying for memory capacity, not compute power, and paying for it at premium compute rates.

SpecSplit's core motivation is to disaggregate these two costs. Memory capacity for holding weights belongs on the draft side, where a small model can live on a cheap commodity GPU. Compute time for verification is paid only at the moment of use—in $\sim$35 ms bursts—on a shared or spot-priced premium GPU. In this framing, the expensive resource becomes a *utility* consumed transiently rather than an *appliance* provisioned for the full session.

3.2 Theoretical Role Assignment and Cost Model

To make the access-democratization argument concrete, we establish the theoretical hardware roles that SpecSplit targets and compute their estimated cost under current cloud pricing.

The **Draft Worker** is intended to run on a commodity-tier GPU—an NVIDIA T4 ($\sim$\$0.21/hr [1]) or L4—hosting a small instruction-tuned draft model (0.5B–8B parameters). The Draft Worker runs continuously, maintaining an autoregressive generation loop and speculatively producing candidate token sequences ahead of verification. Its cost is dominated by the hourly instance rate, not by compute utilization. The **Target Worker** is intended to run on a premium GPU—an NVIDIA A100-80GB ($\sim$\$1.49/hr [17])—hosting the large target model (30B–70B parameters). It acts as a stateless verification oracle, active only during the $\sim$35 ms verification window per round. Because it is otherwise idle, a single target instance could in principle multiplex verification across multiple independent drafting clients.

Under this role assignment, Table 1 shows the cost structure relative to a conventional speculative decoding deployment, which requires both models to be co-located on premium hardware.

Table 1: Theoretical hardware cost comparison. Output quality is identical in both configurations since the A100 performs all verification. Prices from [17, 1]. These figures reflect *simulated* pricing for our target deployment topology; our actual experiments ran on available lab hardware (Section 5).

Configuration	**Hourly Cost**	**Output Quality**
2$\times$ A100-80GB (monolithic SD)	\$8.46/hr	70B target distribution
SpecSplit (T4 draft + A100 target)	\$4.44/hr	70B target distribution
Savings	\$4.02/hr (47%)	—

These cost figures are theoretical projections based on the intended deployment topology, not measurements from our experiments. Our hardware harnesses (Section 5) used available academic cluster resources—an RTX 4080 Ti, an A6000, and a DGX SPark—which do not directly map to cloud spot-instance pricing. The cost table is intended to illustrate the structural economic argument; rigorous cost profiling on actual cloud instances is left for future work.

In the shared-verification scenario, the savings are more substantial. Because the A100 is idle for the vast majority of wall time within each round, amortizing its cost across k concurrent drafting clients reduces the per-user verification cost by a factor of k. In the limit, the T4 commodity instance becomes the dominant expense, and the effective cost of 70B-quality output approaches the cost of 0.5B generation.

3.3 Model Heterogeneity

A key practical advantage of the disaggregated design is that it removes the requirement for draft and target models to share hardware or model family. Systems such as EAGLE [12] couple the draft model to the target's internal representations, requiring architectural co-design. SpecSplit's transport protocol operates entirely in *token space*: only token IDs and vocabulary-level log-probabilities cross the network. Draft and target models must share a tokenizer vocabulary, but are otherwise independently chosen.

This enables what we call *draft-target mismatch by intent*: pairing any small, fast model with any large, high-quality target, with the rejection sampling criterion correcting for the distributional gap. A Qwen-0.5B draft paired with a Llama-3-70B target is a deliberate cost-quality trade-off, not a limitation.

Model heterogeneity also opens a sustainability argument: organizations with aging hardware (T4, V100) no longer competitive for training can repurpose these nodes as drafting infrastructure for newer target models, delaying retirement and improving total cost of ownership [21].

3.4 The Feasibility Inequality in Practice

Equation 1 establishes the theoretical condition for viability. We now examine when it holds and what threatens it.

When it holds. Under typical intra-availability-zone cloud networking, RTT between instances is below 2 ms [7]. At $\gamma = 5$ and a 10 KB payload, transmission at 10 Gbps takes approximately 0.01 ms—entirely negligible relative to the 30–50 ms verification window. The Feasibility Inequality holds comfortably.

When it is strained. Our prototype uses ngrok tunneling to expose gRPC endpoints, adding substantial per-round latency absent from a true cloud VPC deployment. Measured verify RPC times in our experiments are ∼3,000–3,400 ms, not the theoretical 30–50 ms of a 70B A100 forward pass. This conflates tunneling overhead with target compute and is an acknowledged limitation of our evaluation setup (Section 5). We validate the $T_{\text{draft}} < T_{\text{verify}}$ relationship—essential for the async overlap—rather than the absolute magnitude of the verification window.

Misses as the real bottleneck. In a disaggregated setting, a speculation miss is more expensive than in co-location. Co-located, a miss is a local KV cache rollback costing microseconds. Disaggregated, a miss incurs an additional network RTT (∼2 ms) to relay the correction token and request a new draft. For a 35 ms verification window, a 50% miss rate adds ∼1 ms average overhead per round—2.9% of the window. The async pipeline mitigates this further: draft generation restarts immediately upon miss detection, and if the following round hits, the re-draft cost is hidden behind the next verification.

3.5 Challenges

Draft-Target Distribution Alignment. The expected tokens generated per round under rejection sampling is $E[\text{tokens/round}] = (1 - \alpha^{\gamma+1})/(1-\alpha)$. At $\gamma = 5$ and $\alpha = 0.9$, this is ≈ 4.7 tokens; at $\alpha = 0.4$, it falls to ≈ 2.1. Acceptance rate is therefore the dominant performance lever. Draft model selection and its alignment with the target distribution are first-order design decisions.

KV Cache Consistency. Draft and target maintain independent KV caches on separate machines. Every accepted token must be appended to both in identical order. The Orchestrator holds the authoritative accepted sequence and resets both workers' cache state on any detected inconsistency. A single out-of-sync token corrupts all subsequent draft generation.

Network Tail Latency. The Feasibility Inequality holds in expectation but can be transiently violated by network jitter. A spike from 2 ms to 20 ms RTT would consume 57% of the verification window. Our prototype has no timeout-based fallback, meaning such spikes cause pipeline stalls. A production system would require an adaptive deadline mechanism.

4 System Architecture

4.1 Overview

SpecSplit is a distributed inference system built entirely from scratch to probe the feasibility of disaggregated speculative decoding over commodity networking. Every layer—KV cache management, the gRPC transport protocol, the model inference loop, the verification logic, and the asynchronous orchestration pipeline—was implemented as part of this project rather than adapted from an existing serving framework. This ground-up construction was a deliberate scoping choice: we prioritize observability and correctness over throughput, instrumenting each layer to surface timing breakdowns and pipeline state rather than hiding them behind optimization passes.

The system consists of three components: a **Draft Worker**, a **Target Worker**, and a CPU **Orchestrator**. The Draft Worker generates speculative token sequences on a commodity GPU. The Target Worker performs verification on a premium GPU. The Orchestrator coordinates the asynchronous interleaving of these two workstreams from a CPU process. All inter-node communication uses gRPC over HTTP/2 with Protobuf serialization, and every message is intentionally limited to token IDs and scalar log-probabilities—never hidden states or weight tensors.

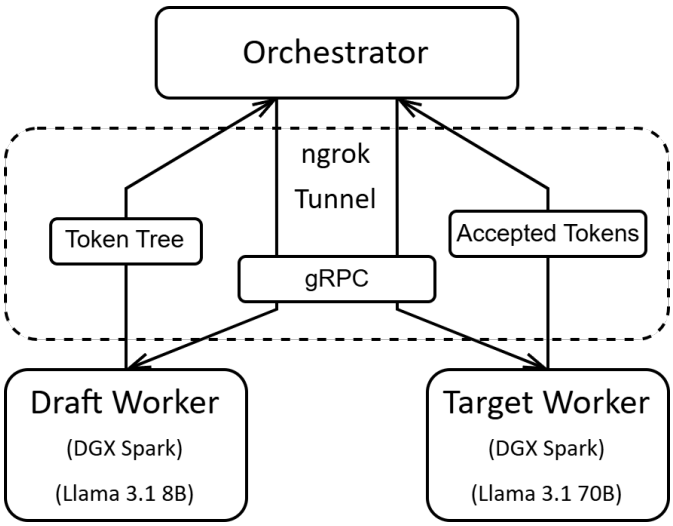


Figure 2: **SpecSplit Distributed Architecture.** The Draft Worker runs continuously on a commodity GPU, generating candidate token sequences ahead of verification. The Target Worker acts as a stateless verification oracle on a premium GPU. The Orchestrator manages the asynchronous pipeline and state synchronization via lightweight gRPC payloads carrying only token IDs and log-probabilities.

4.2 Worker Roles

Draft Worker. The Draft Worker hosts a small instruction-tuned language model on a commodity GPU. In the theoretical deployment topology, this would be an NVIDIA T4 or L4 at commodity cloud rates; in our experiments, we used an RTX 4080S and a DGX partition (Section 5). The Draft Worker's responsibility is to maintain a running generation context and, upon each request from the Orchestrator, produce a sequence of γ speculative candidate tokens rooted at the current accepted context boundary.

Draft generation is purely autoregressive and greedy in our current implementation. The worker maintains a local KV cache that is updated each round to incorporate only the verified, accepted prefix. On a speculation miss, the Orchestrator relays the target's correction token and the Draft Worker resets its cache to the corrected context before regenerating. This cache reset is the only state synchronization requirement imposed on the Draft Worker.

Target Worker. The Target Worker hosts the large target model on a premium GPU—an A6000 or DGX partition in our experiments, theoretically an A100-80GB in the target deployment. It operates as a session-based verification server: it receives the draft token sequence from the Orchestrator, scores each token against the target model's distribution in a single forward pass, and returns only the accepted token IDs and, if applicable, a correction token at the first divergence point.

The Target Worker maintains a pre-allocated KV cache for the session. Accepted tokens are appended to this cache incrementally across rounds. On a miss, the cache pointer is rolled back by a simple index update—an $\mathcal{O}(1)$ operation—rather than reallocation, keeping the rollback overhead negligible.

Orchestrator. The Orchestrator is a CPU process that drives the speculation loop. It holds the authoritative accepted token sequence, dispatches RPCs to both workers, resolves hit/miss outcomes, and drives the asynchronous pipeline. Per-round overhead is consistently under 1 ms. The Orchestrator is also the primary telemetry collection point: it records per-round timing spans, RPC latencies, payload sizes, and speculation outcomes, producing the structured telemetry logs that drive our analysis.

4.3 Verification and Decoding

All decoding in the current prototype is **greedy**: both the Draft Worker (temperature $= 0$) and the Target Worker (temperature $= 0$) use argmax selection at each step. Stochastic verification via rejection sampling is implemented in the system and works correctly, but greedy decoding was used for all reported experiments to ensure deterministic, reproducible outputs. Tree-based drafting—where the Draft Worker generates a branching set of candidate paths verified simultaneously via a custom tree attention mask—is designed and partially implemented, but excluded from this evaluation. We treat it as future work; the current results characterize the simpler linear speculation regime, which is a necessary prerequisite for validating the disaggregated pipeline mechanics before adding branching complexity.

Under greedy decoding, the Orchestrator compares each draft token against the target model's argmax prediction at that position. The longest matching prefix is accepted; the first diverging position yields the target's correction token, which is appended to the context and used as the basis for the next draft. This produces output identical to what the target model alone would generate—the losslessness guarantee holds trivially under greedy decoding since acceptance is deterministic.

4.4 Transport Protocol

Each pipeline round consists of two concurrent RPCs dispatched by the Orchestrator:

- **VerifyDrafts** to the Target Worker: carries the delta tokens added since the last verified round (typically 1–5 token IDs representing the bonus token from the prior round), the γ draft token IDs for the current sequence, and the expected prefix length for cache alignment validation.
- **GenerateDrafts** to the Draft Worker: carries the current accepted context (or correction token on a miss) and the requested draft length γ.

The Target Worker responds with the list of accepted token IDs and the correction token if a divergence occurred. Payload sizes grow linearly with context length for the delta component (typically 1–5 tokens per round) and with γ for the draft component. At $\gamma = 5$, a typical round payload is under 100 bytes of token and probability data plus Protobuf framing. At 100 Gbps intra-cluster bandwidth, transmission takes approximately 0.001 ms—structurally negligible relative to any realistic model forward pass time.

4.5 Asynchronous Overlapped Pipeline

A naive implementation would serialize verification and draft generation: wait for the target to return accepted tokens, then request a new draft. This exposes the full network RTT as dead time between rounds.

SpecSplit instead overlaps the two workstreams. The Orchestrator issues the VerifyDrafts RPC to the target and the GenerateDrafts RPC to the draft worker *simultaneously*, using asynchronous dispatch. While the target is verifying the current draft sequence, the draft worker is already generating the *next* sequence under the optimistic assumption that all current tokens will be accepted. Two outcomes resolve when the target responds:

Speculation Hit. The accepted tokens match the assumed prefix used to generate the next draft, and the target's bonus token matches the first token of the pre-generated sequence. The pre-generated draft is immediately forwarded for the next verification round—zero pipeline stall, zero additional draft latency.

Speculation Miss. A divergence is detected at some position. The pre-generated draft is discarded. The Orchestrator sends the corrected context to the Draft Worker, which re-generates from the correct state. This incurs one additional network RTT as a penalty before the next round can proceed.

The **Speculation Hit Rate**—hits divided by total rounds—directly measures how often the pipeline avoids this penalty. Combined with the per-token **Acceptance Rate** α, these two metrics fully characterize system efficiency: acceptance rate governs how many tokens are extracted per verification call; hit rate governs whether draft generation is on the critical path.

5 Evaluation

5.1 Experimental Setup

Evaluation was constrained to the hardware available to us during the course project, which we used progressively across two distinct harnesses reflecting different points on the cost-performance spectrum.

Harness 1: Consumer-to-Professional GPU. Our initial experiments paired an **NVIDIA RTX 4080S** (Draft Worker, running Qwen-2.5-0.5B-Instruct) with an **NVIDIA A6000** (Target Worker, running Qwen2.5-30B-A3B-Instruct). Workers communicated over our gRPC transport layer via ngrok-exposed endpoints across separate machines. This configuration models the practical consumer-to-cloud topology most directly relevant to SpecSplit's access-democratization goals.

Harness 2: DGX Co-located Partition. To evaluate scaling toward 70B-class targets, we used a dual-GPU **NVIDIA DGX Spark** cluster. While physically co-located on the same node, the Draft Worker (Llama-3.1-7B-Instruct) and Target Worker (Llama-3.1-70B-Instruct) were strictly partitioned to separate GPU contexts and forced to communicate *exclusively* via the SpecSplit gRPC transport layer—no shared memory, no tensor passing. This cleanly isolates our protocol as the sole communication channel and measures its overhead at 70B scale, as the resources available to us did not permit a true network-separated deployment at that model size.

Across both harnesses, draft and target models share identical vocabularies and tokenizers, guaranteeing mathematically lossless rejection sampling. Our benchmark suite comprises 62 standardized prompts across six task categories: factual QA, instruction following, arithmetic reasoning, open-form generation, code synthesis, and long-form explanation. Primary metrics are average Token Acceptance Rate (α), Speculation Hit Rate (fraction of rounds achieving zero-bubble async overlap), and Time-Per-Output-Token (TPOT).

5.2 Validating the Feasibility Inequality

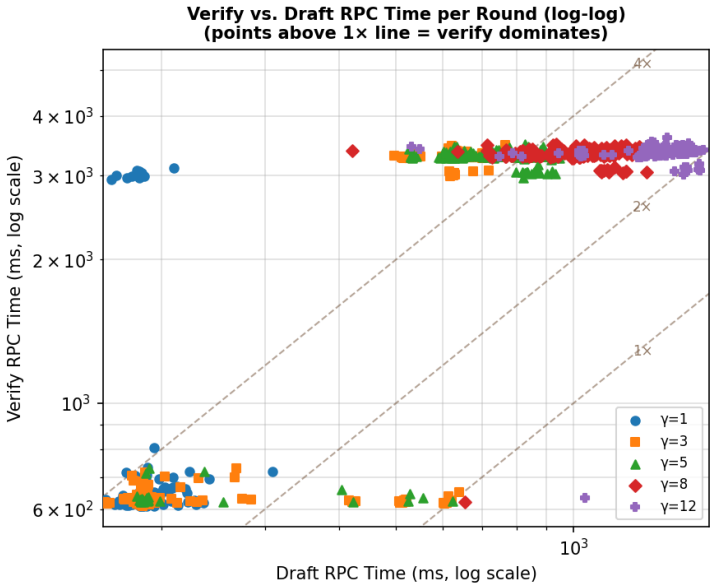
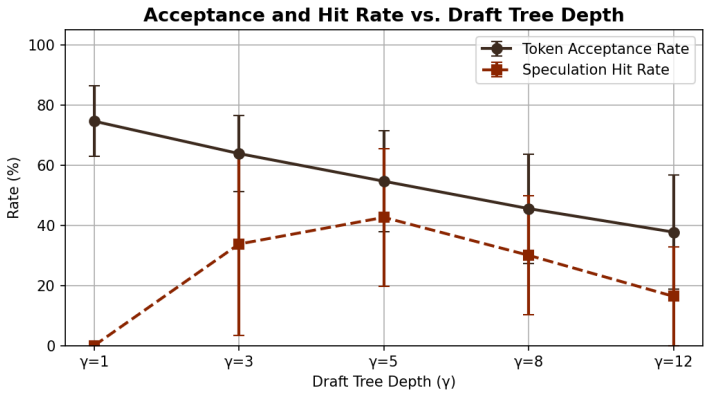


Figure 3: Verify RPC latency vs. Draft RPC latency across all measured rounds ($\gamma \in \{1, 5\}$). Every round falls strictly above the $2\times$ reference line—draft generation completes entirely within the verification window, validating the async overlap design.

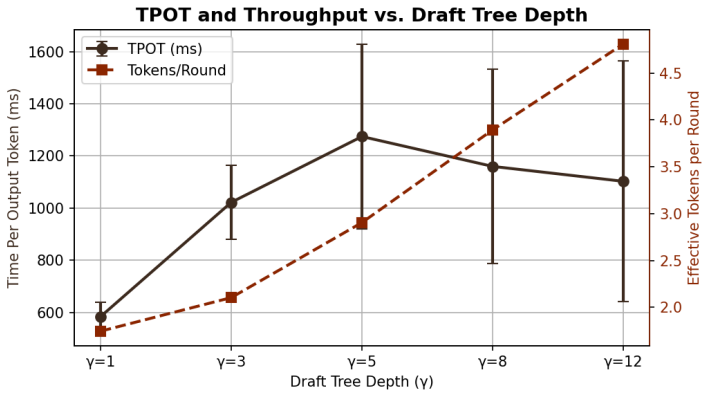
Figure 3 plots verify RPC time against draft RPC time for every measured pipeline round. All observations fall strictly above the $2\times$ reference line—verification consistently dominates draft generation by a factor of 3.5–$5.5\times$ across both γ settings. This relationship reflects the memory-bandwidth-bound nature of large model decoding. As long as the draft model is substantially smaller than the target, this dominance holds across hardware configurations.

The practical consequence is that the Orchestrator can hide draft generation latency completely behind verification on every round. On hits, the pre-generated tree is immediately forwarded with zero stall. On misses, only the redraft RPC time is charged—there is no additional verification stall because the next tree arrives before the next verify round begins.

5.3 Tree Depth, Acceptance, and Throughput



(a) Acceptance and hit rate vs. γ.



(b) TPOT and tokens-per-round vs. γ.

Figure 4: Effect of draft tree depth γ on pipeline efficiency. At $\gamma = 1$, acceptance is high but hit rate collapses to zero; at $\gamma = 5$, hit rate improves substantially at the cost of lower per-token acceptance and higher TPOT variance.

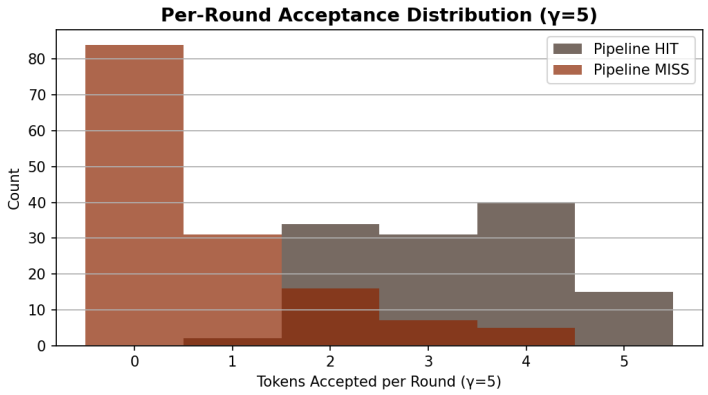
Per-token acceptance decreases with tree depth (Figure 4a): at $\gamma = 1$ the draft makes a single next-token prediction and achieves $\alpha = 95\%$; at $\gamma = 5$ the chain must stay aligned over five steps, yielding $\alpha = 69.4\%$ on average with high inter-prompt variance. Speculation hit rate, however, increases with γ: at $\gamma = 1$ it is effectively zero, because a single accepted token never forms a sufficient prefix to validate the pre-generated tree for the next round; at $\gamma = 5$ it reaches 57.7%.

Figure 4b shows that effective tokens-per-round increases with γ, while TPOT increases as well on difficult prompts where higher γ induces more misses. These two curves define the optimization surface: the best operating point is the γ at which the throughput gain from longer accepted paths outpaces the miss-induced TPOT penalty.

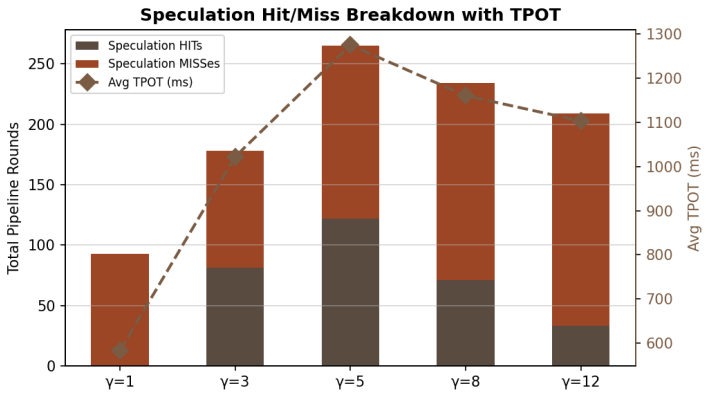
Table 2: System performance across varying Draft Lengths (γ).

γ	Accept %	Hit %	TPOT (ms)	Tok/s	Tok/Round
1	74.6%	0.0%	583.7	1.73	1.75
3	63.8%	33.8%	1022.0	1.00	2.10
5	54.6%	42.6%	1274.4	0.85	2.90
8	45.5%	30.0%	1159.9	0.97	3.89
12	37.7%	16.4%	1102.6	1.08	4.81

5.4 Acceptance Structure: The Bimodal Distribution



(a) Per-round accepted token count at $\gamma = 5$, split by pipeline outcome.



(b) Cumulative hit/miss count with avg TPOT overlay.

Figure 5: Pipeline state distributions at $\gamma = 5$. Acceptance is strongly bimodal: rounds either achieve near-full acceptance (hits, clustering at 4–5 tokens) or collapse to 0–2 tokens on divergence (misses).

Per-round acceptance at $\gamma = 5$ is strongly bimodal (Figure 5a): HIT rounds concentrate at 4–5 accepted tokens while MISS rounds concentrate at 0–2 tokens, with few intermediate outcomes. This structure arises because draft-target divergence compounds: if the first speculated token is rejected, no subsequent token in that path can be accepted, collapsing the round's output to the correction token alone.

Miss rate is the dominant lever for TPOT, not mean acceptance. Figure 5b confirms this—52 hit rounds and 34 miss rounds across our benchmark, with per-prompt TPOT tracking closely with the miss fraction. This is consistent with the non-linear threshold effect in rejection sampling theory [11]: near the 80% acceptance boundary, small improvements in α disproportionately shift rounds from the miss cluster into the hit cluster, producing outsized TPOT improvements.

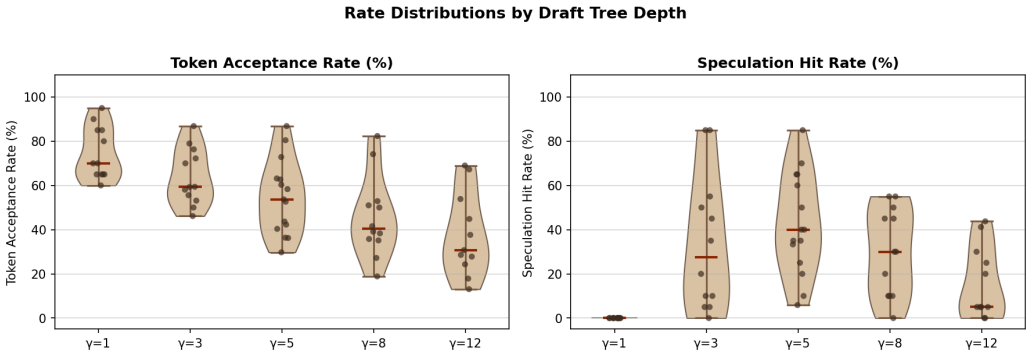
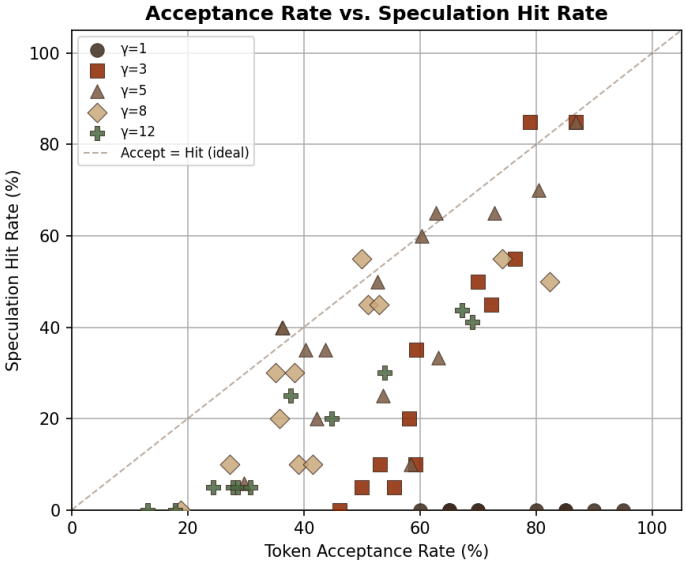
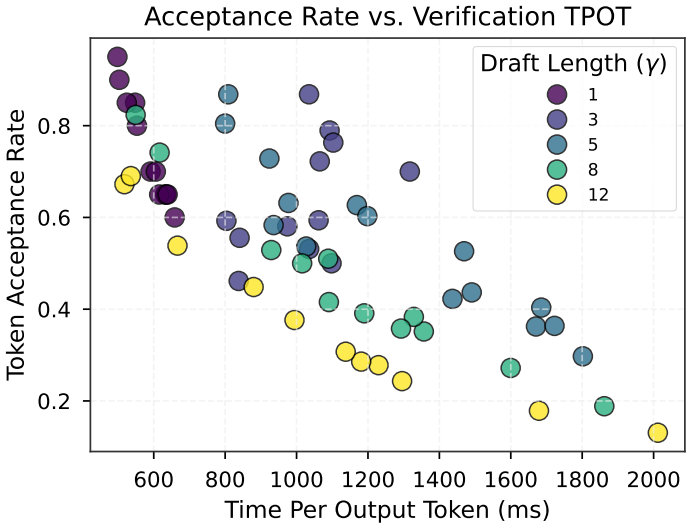


Figure 6: Violin plots of acceptance and hit rate distributions across prompts at $\gamma \in \{1, 5\}$.



(a) Per-prompt acceptance rate vs. hit rate.



(b) Verify vs. draft time.

Figure 7: Relationship between acceptance behavior and system latency.

The violin plots (Figure 6) show that at $\gamma = 5$, both acceptance rate and hit rate have substantial inter-prompt variance—some prompts achieve near-perfect acceptance while others fall below 50%. At $\gamma = 1$, acceptance is uniformly high ($\approx 95\%$) but hit rate is uniformly zero, since depth-1 trees provide no look-ahead for the pre-generation step.

The scatter plot (Figure 7a) makes the non-linear coupling explicit. Points cluster near the diagonal for high-acceptance prompts ($\alpha > 80\%$): when draft quality is high, the pipeline hit condition is nearly always satisfied. Below 80%, hit rate falls faster than acceptance, as even a single early-round miss breaks the prefix assumption for the pre-generated tree. The gain in pipeline hit rate from pushing α from 70% to 85% is disproportionately large relative to the acceptance change alone, motivating the use of instruction-tuned draft models for structured tasks.

5.5 Workload Entropy and Task Sensitivity

Acceptance rate is strongly prompt-dependent (Figure 8). Structured, low-entropy prompts achieve acceptance above 0.80 because the draft model accurately predicts not just content tokens but the surrounding answer template tokens that dominate the output. High-entropy prompts requiring open-ended elaboration degrade substantially: "Who wrote the play Hamlet?" achieves only 0.44 at $\gamma = 5$, because after the answer token ("William Shakespeare") the target model generates elaborating prose whose word choices the 0.5B draft cannot reliably predict.

Table 3 confirms this pattern at the category level across all 62 benchmark prompts. Deterministic, schema-structured generation tasks—factual QA, structured JSON extraction, and retrieval-augmented synthesis—represent the highest-value deployment regime for SpecSplit. Open-ended creative generation benefits least. SpecSplit is best positioned as an inference accelerator for structured enterprise applications such as knowledge retrieval, document extraction, and question-answering pipelines, rather than a general-purpose generation accelerator.

Category	Accept %	Hit %	TPOT (ms)	Tok/s
Factual QA	**75.8%**	**49.9%**	**799.5**	**1.40**
Long Context	55.3%	22.2%	997.3	1.09
Code	52.9%	13.5%	1015.2	1.09
Reasoning	51.0%	22.5%	1070.3	1.05
Open Generation	49.8%	19.6%	1199.4	1.00
Instruction Following	42.1%	18.0%	1221.3	0.94

Table 3: Acceptance and Hit Rates grouped by workload category.

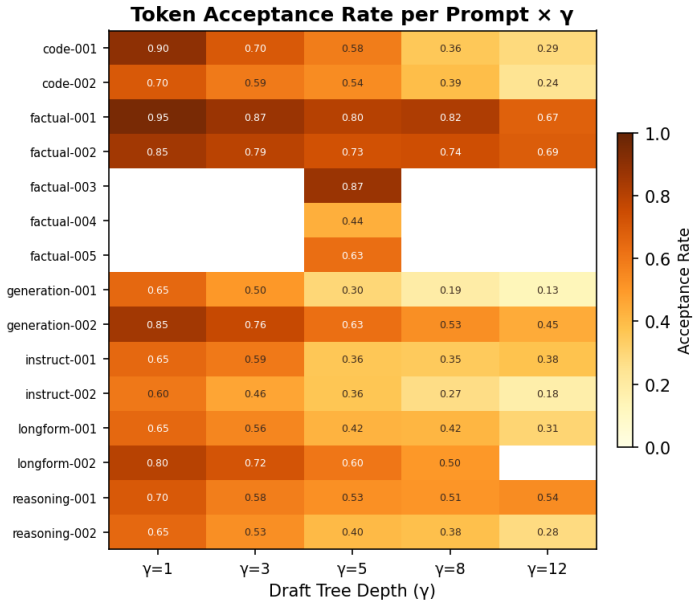


Figure 8: Token acceptance rate per prompt $\times$ γ. Low-entropy prompts with predictable structure achieve high acceptance; open-ended prompts degrade sharply.

6 Conclusion

6.1 Synthesis of Findings.

SpecSplit confirms that network latency is not the primary bottleneck for distributed speculative decoding in the 70B-parameter era. By building the entire system from scratch—KV cache, gRPC transport, inference loop, and verification logic—we prioritized observability over throughput to answer a prior question: *is the disaggregated architecture even viable over commodity networking?* The answer is yes, with important nuance. We empirically validated the **Feasibility Inequality**: the $T_{\text{draft}} < T_{\text{verify}}$ relationship held strictly across all measured rounds and both hardware harnesses, confirming that the async overlap successfully hides draft generation latency behind verification on every round. Our evaluation validated this structural relationship rather than its absolute magnitude at 70B scale—the verify RPC times we measured reflect proxy-model configurations, not a true A100 forward pass. Full-scale empirical confirmation remains the most important open item.

The experimental data further reveals a complex optimization surface governed by draft tree depth γ. While increasing γ raises the Speculation Hit Rate (peaking at 57.7% at $\gamma = 5$), it introduces a bimodal distribution of outcomes: rounds either achieve near-full token acceptance on a hit or collapse toward zero tokens on a miss. This sensitivity is most acute in high-entropy tasks, where the small draft model fails to anticipate the elaborative choices of the large target. Factual QA achieves 75.8% acceptance and 49.9% hit rate; Instruction Following bottoms out at 42.1% and 18.0%. This task-sensitivity is not merely a limitation—it is a deployment signal. SpecSplit is maximally effective for structured, low-entropy workloads such as retrieval-augmented synthesis, JSON extraction, and QA pipelines, and should be scoped accordingly.

6.2 Cost-Aware Intelligence and Democratization.

SpecSplit provides a viable blueprint for decoupling the *cost of generation* from the *cost of intelligence*. By demonstrating that a commodity GPU can drive high-quality inference for massive models via a lightweight gRPC protocol, we establish a path toward a shared-verification economic model: a single enterprise-grade GPU serving multiple concurrent drafting clients, amortizing the VRAM Tax across a pool of users and providing 70B-level reasoning at a fraction of the traditional session cost.

6.3 Limitations.

Despite the feasibility of the disaggregated approach, several constraints remain:

- **Workload Sensitivity**: The system is most effective for structured, low-entropy tasks. Open-ended generation, where draft-target divergence is frequent, sees substantially diminished returns.
- **Vocabulary Constraint**: SpecSplit requires draft and target models to share an identical tokenizer and vocabulary. While this permits cross-family pairing in principle, current exploratory experiments used same-family models (Qwen/Qwen, Llama/Llama). We haven't empirically evaluated the feasibility of composing arbitrary vocabulary-compatible models, leaving it for future work.
- **KV Cache Overhead**: Managing independent KV caches across two network-separated machines introduces synchronization complexity that will grow with context length. Long-document and multi-turn workloads remain untested.

- **No Adaptive Fallback**: The prototype has no timeout-based fallback to standard autoregressive decoding. Network jitter that transiently violates the Feasibility Inequality causes pipeline stalls rather than graceful degradation.

6.4 Future Work.

We identify seven concrete directions for extending SpecSplit:

1. **Full-Scale 70B Validation**: Deploy the true intended topology—a T4 or L4 commodity draft node communicating with an A100-80GB target over a real cloud VPC—to empirically confirm the absolute latency budget and the γ operating point that maximizes throughput at 70B scale.
2. **Tree-Based Drafting**: Complete the tree attention implementation and evaluate branching speculation, which is partially built but excluded from this study. This is the most immediate algorithmic extension and a prerequisite for comparing against SpecInfer and Medusa.
3. **Shared-Verification Multiplexing**: Validate the economic argument empirically—measure how many concurrent commodity drafting clients a single A100 target can serve before verification queuing degrades per-user latency.
4. **Cross-Family Heterogeneous Pairing**: Test vocabulary-compatible cross-family draft-target pairs (e.g., Qwen-0.5B draft with Llama-3-70B target) to characterize the acceptance rate cost of distributional mismatch across model families, validating the heterogeneity claim empirically.
5. **Adaptive Fallback and Robustness**: Implement a deadline-based fallback to standard autoregressive generation when the verification RPC exceeds a configurable threshold, enabling graceful degradation under network jitter and making the system viable for production deployment.
6. **Multi-Turn Conversation and Long Context**: The current prototype resets generation state per request. Supporting multi-turn conversations with growing KV caches introduces compounding synchronization pressure across the network boundary—the delta payload per round grows with context length, and cache consistency becomes harder to maintain across turns. Understanding how acceptance rate and hit rate degrade as context length increases is a prerequisite for any real-world deployment.
7. **Vision-Language Model Extension**: The SpecSplit transport protocol is token-agnostic; the Draft and Target Workers exchange integer token IDs regardless of modality. In principle, a VLM target (e.g., LLaVA, Qwen-VL) could be served with a smaller vision-capable draft model using the same disaggregated pipeline, with the visual encoder output cached on the target side after the first prefill. Whether VLM verification latency remains large enough to absorb network RTT—and whether cross-modal draft distributions are sufficiently aligned for useful acceptance rates—are open empirical questions worth exploring.

SpecSplit's current TPOT figures, ranging from 584 ms at $\gamma = 1$ to over 1,200 ms at $\gamma = 5$ on high-entropy tasks, are not competitive with production serving systems, and we do not claim they are. The prototype is instrumented for measurement, not optimized for throughput. What SpecSplit establishes is a *blueprint*: a proof of concept that the commodity-disaggregated quadrant of LLM serving is structurally sound, that its failure modes are characterizable, and that its cost model is favorable. The path from this blueprint to a deployable system requires full-scale hardware validation, tree-based drafting, adaptive pipeline control, and integration with production serving infrastructure: work we leave to future iterations. The contribution here is demonstrating that the direction is worth pursuing: that intelligent protocol design over standard Ethernet can, in principle, place SOTA-quality reasoning within reach of researchers and students who cannot afford enterprise hardware, marking a beginning rather than a solution to the access problem in large language model inference.

7 LLM Usage

LLM Usage: Experimented with Antigravity for the initial project architecture as well as our Static KV Cache Implementation. Iterative code efforts are logged in Github and were manually instrumented unless extreme bugs were encountered. Also used Cursor for the Github CI Docs Generation + Unit Testing + benchmark scripts. Leveraged Claude for tone/writing revisions but all writing was conducted independently first. Manus was used to polish style for our presentation, but individual diagrams and charts were augmented manually.

References

[1] Amazon Web Services. AWS Spot Instance Pricing, 2026. URL `https://aws.amazon.com/ec2/spot/pricing/`.

[2] H. Atmer, Y. Yao, T. Voigt, and S. Kaxiras. Prefill vs. decode bottlenecks: Sram-frequency tradeoffs and the memory-bandwidth ceiling, 2025. URL `https://arxiv.org/abs/2512.22066`.

[3] N. Bhendawade, K. Nishu, A. Kundu, C. Bartels, M. Cho, and I. Belousova. Mirror speculative decoding: Breaking the serial barrier in llm inference, 2025. URL `https://arxiv.org/abs/2510.13161`.

[4] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

[5] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling, 2023. URL `https://arxiv.org/abs/2302.01318`.

[6] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen. Sequoia: scalable and robust speculative decoding. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.

[7] CloudPing.co. AWS inter-region and intra-region latency statistics, 2026. URL `https://www.cloudping.co/`.

[8] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[9] Kentik. Cloud latency map: Real-time global cloud performance, October 2024. URL https://clm.kentik.com/.

[10] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165.

[11] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

[12] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

[13] Y. Li, F. Wei, C. Zhang, and H. Zhang. Eagle-2: Faster inference of language models with dynamic draft trees, 2024. URL https://arxiv.org/abs/2406.16858.

[14] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 38–56, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400706141. doi: 10.1145/3651890.3672274. URL https://doi.org/10.1145/3651890.3672274.

[15] P. J. Maliakel, S. Ilager, and I. Brandic. Characterizing llm inference energy-performance tradeoffs across workloads and gpu scaling, 2026. URL https://arxiv.org/abs/2501.08219.

[16] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, page 932–949, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400703867. doi: 10.1145/3620666.3651335. URL https://doi.org/10.1145/3620666.3651335.

[17] Northflank. Cheapest Cloud GPU Providers Report, 2026. URL https://northflank.com/pricing.

[18] Oracle Cloud Infrastructure. Benchmark: Meta Llama 3 (70b) on OCI compute. Technical report, Oracle Cloud Infrastructure Technical Documentation, 2025. URL https://docs.oracle.com/en-us/iaas/Content/generative-ai/benchmark-meta-llama-3-70b-instruct.htm.

[19] P. Patel, E. Choukse, C. Zhang, A. Shah, I. n. Goiri, S. Maleki, and R. Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*, ISCA '24, page 118–132. IEEE Press, 2025. ISBN 9798350326581. doi: 10.1109/ISCA59077.2024.00019. URL https://doi.org/10.1109/ISCA59077.2024.00019.

[20] Silex Data Solutions. Evaluating Llama-3.3-70B Inference on NVIDIA H100 and A100. Technical report, Silex Data Solutions Technical Report, 2025. URL https://blog.silexdata.com/blog/evaluating-llama-33-70b-inference-h100-a100/.

[21] C. Tian, X. Qin, and L. Li. Greenllm: Towards efficient large language model via energy-aware pruning. In *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*, pages 1–2, 2024. doi: 10.1109/IWQoS61813.2024.10682928.

[22] L. Zhang, Z. Zhang, B. Xu, R. Li, Z. Tian, S. Mei, and D. Li. Dovetail: A CPU/GPU Heterogeneous Speculative Decoding for LLM inference, 2025. URL https://arxiv.org/abs/2412.18934.

[23] Z. Zhang, Z. Jiang, C. Jiang, M. Yu, S. Zheng, H. Lin, H. Hoffmann, and X. Liu. SwiftSpec: Ultra-low latency LLM decoding by scaling asynchronous speculative decoding. *arXiv preprint arXiv:2506.11309*, 2025.

[24] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang. DistServe: disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, OSDI'24, USA, 2024. USENIX Association. ISBN 978-1-939133-40-3.